

# define\_static\_{string,object,array}

Document #: P3491R3 [[Latest](#)] [[Status](#)]  
Date: 2025-06-20  
Project: Programming Language C++  
Audience: CWG, LWG  
Reply-to: Wyatt Childers  
<[wcc@edg.com](mailto:wcc@edg.com)>  
Peter Dimov  
<[pdimov@gmail.com](mailto:pdimov@gmail.com)>  
Barry Revzin  
<[barry.revzin@gmail.com](mailto:barry.revzin@gmail.com)>  
Daveed Vandevoorde  
<[daveed@edg.com](mailto:daveed@edg.com)>

## Contents

---

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Proposal](#)
  - [3.1 The Overlapping Question](#)
    - [3.1.1 Update from CWG](#)
  - [3.2 The Structural Question](#)
  - [3.3 Return Type and Layering](#)
  - [3.4 Possible Implementation](#)
  - [3.5 Examples](#)
    - [3.5.1 Use as non-type template parameter](#)
    - [3.5.2 Pretty-printing](#)
    - [3.5.3 Promoting Containers](#)
    - [3.5.4 With Expansion Statements](#)
    - [3.5.5 Implementing source location](#)
  - [3.6 Related Papers in the Space](#)
- [4 Wording](#)
  - [4.1 Feature-Test Macro](#)
- [5 Acknowledgements](#)
- [6 References](#)

## § 1 Revision History

---

Since [\[P3491R2\]](#), merged in [\[P3617R0\]](#) `(std::meta::reflect_constant_{array,string})` following LEWG telecon on May 20th.

Since [\[P3491R1\]](#), added support for all string literal types. Having discovered [\[CWG2765\]](#) `(Address comparisons between potentially non-unique objects during constant evaluation)`, referring to that issue and updating wording to assume its adoption. Retargeting CWG and LWG.

Since [\[P3491R0\]](#), wording improvements.

## § 2 Introduction

---

These functions were originally proposed as part of [\[P2996R7\]\\_\(Reflection for C++26\)](#), but are being split off into their own paper.

There are situations where it is useful to take a string (or array) from compile time and promote it to static storage for use at runtime. We currently have neither:

- non-transient constexpr allocation (see [\[P1974R0\]\\_\(Non-transient constexpr allocation using \\_propconst\)](#), [\[P2670R1\]\\_\(Non-transient constexpr allocation\)](#)), nor
- generalized support for class types as non-type template parameters (see [\[P2484R0\]\\_\(Extending class types as non-type template parameters\)](#), [\[P3380R1\]\\_\(Extending support for class types as non-type template parameters\)](#))

If we had non-transient constexpr allocation, we could just directly declare a static constexpr variable. And if we could use these container types like `std::string` and `std::vector<T>` as non-type template parameter types, then we would use those directly too.

But until we have such a language solution, people have over time come up with their own workarounds. For instance, Jason Turner in a [recent talk](#) presents what he calls the “constexpr two-step.” It’s a useful pattern, although limited and cumbersome (it also requires specifying a maximum capacity).

Similarly, the lack of general support for non-type template parameters means we couldn’t have a `std::string` template parameter (even if we had non-transient constexpr allocation), but promoting the contents of a string to an external linkage, static storage duration array of `const char` means that you can use a pointer to that array as a non-type template parameter just fine.

So having facilities to solve these problems until the general language solution arises is very valuable.

## § 3 Proposal

This paper proposes several new library functions, grouped into three kinds:

- helper function for identifying string literals
- three higher-level functions for defining static storage objects: `define_static_string`, `define_static_object`, and `define_static_array`. These are just in `std` as their interface is not strictly reflection-related
- two lower-level functions for returning a reflection of an array: `meta::reflect_constant_string` and `meta::reflect_constant_array` (from [\[P3617R0\]](#)).

```
namespace std {
    constexpr auto is_string_literal(char const* p) -> bool;
    constexpr auto is_string_literal(wchar_t const* p) -> bool;
    constexpr auto is_string_literal(char8_t const* p) -> bool;
    constexpr auto is_string_literal(char16_t const* p) -> bool;
    constexpr auto is_string_literal(char32_t const* p) -> bool;

    namespace meta {
        template <ranges::input_range R> // only if the value_type is char or char8_t
        constexpr auto reflect_constant_string(R&& r) -> info;

        template <ranges::input_range R>
        constexpr auto reflect_constant_array(R&& r) -> info;
    }

    template <ranges::input_range R> // only if the value_type is char or char8_t
    constexpr auto define_static_string(R&& r) -> ranges::range_value_t<R> const*;

    template <class T>
    constexpr auto define_static_object(T&& v) -> remove_reference_t<T> const*;
```

```
template <ranges::input_range R>
constexpr auto define_static_array(R&& r) -> span<ranges::range_value_t<R> const>;
}
```

`is_string_literal` takes a pointer to either `char const` or `char8_t const`. If it's a pointer to either a string literal `V` or a subobject thereof, these functions return `true`. Otherwise, they return `false`. Note that we can't necessarily return a pointer to the start of the string literal because in the case of overlapping string literals — how do you know which pointer to return?

`define_static_string` is limited to ranges over `char` or `char8_t` and returns a `char const*` or `char8_t const*`, respectively. They return a pointer instead of a `string_view` (or `u8string_view`) specifically to make it clear that they return something null terminated. If `define_static_string` is passed a string literal that is already null-terminated, it will not be doubly null terminated.

`define_static_array` exists to handle the general case for other types, and now has to return a span so the caller would have any idea how long the result is. This function requires that the underlying type `T` be structural.

`define_static_object` is a special case of `define_static_array` for handling a single object. Technically, `define_static_object(v)` can also be achieved via `define_static_array(views::single(v), data())`, but it's has its own use as we'll show.

Technically, `define_static_array` can be used to implement `define_static_string`:

```
constexpr auto define_static_string(string_view str) -> char const* {
    return define_static_array(views::concat(str, views::single('\0'))).data();
}
```

But that's a fairly awkward implementation, and the string use-case is sufficiently common as to merit a more ergonomic solution.

There are three design questions that we have to address: whether objects can overlap, whether `define_static_array` needs to mandate structural, and what the return type should be.

## § 3.1 The Overlapping Question

Consider the existence of `template <char const*> struct C`; and the following two translation units:

| TU #1                                                                                          | TU #2                                                                                           |
|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <pre>C&lt;define_static_string("dedup")&gt; c1; C&lt;define_static_string("dup")&gt; c2;</pre> | <pre>C&lt;define_static_string("holdup")&gt; c3; C&lt;define_static_string("dup")&gt; c4;</pre> |

In the specification in [\[P2996R7\]](#), the results of `define_static_string` were allowed to overlap. That is, a possible result of this program could be:

| TU #1                                                                                                       | TU #2                                                                                                           |
|-------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <pre>inline char const __arr_dedup[] = "dedup"; C&lt;__arr_dedup&gt; c1; C&lt;__arr_dedup + 2&gt; c2;</pre> | <pre>inline char const __arr_holdup[] = "holdup"; C&lt;__arr_holdup&gt; c3; C&lt;__arr_holdup + 3&gt; c4;</pre> |

This means whether `c2` and `c4` have the same type is unspecified. They could have the same type if the implementation chooses to not overlap (or no overlap is possible). Or they could have different types.

However, that's not the right way to think about overlapping.

A more accurate way to present the ability to support overlapping arrays from `define_static_string` would be that the two TUs would merge more like this:

| TU #1                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | TU #2                                                                                                                                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre><i>// all our static strings merged</i> inline constexpr char const __arr_dedup[] = "dedup"; inline constexpr char const __arr_holdup[] = "holdup";  <i>// this behaves like an array for all purposes, including that</i> <i>// __arr_dup[-1] is not a valid constant expression (because out of bounds)</i> <i>// but the implementation is allowed to have &amp;__arr_dup[0] == &amp;__arr_dedup[2]</i> inline constexpr char const __arr_dup[] = "dup";</pre> |                                                                                                                                                               |
| <pre><i>// C&lt;define_static_string("dedup")&gt;</i> C&lt;__arr_dedup&gt; c1;  <i>// C&lt;define_static_string("dup")&gt;</i> C&lt;__arr_dup&gt; c2;</pre>                                                                                                                                                                                                                                                                                                            | <pre><i>// C&lt;define_static_string("holdup")&gt;</i> C&lt;__arr_holdup&gt; c3;  <i>// C&lt;define_static_string("dup")&gt;</i> C&lt;__arr_dup&gt; c4;</pre> |

At this point, the usual template-argument-equivalence rules apply, so `c4` and `c2` would definitely have the same type, because their template arguments point to the same array. As desired.

The one thing we really have to ensure with this route, as pointed out by Tomasz Kamiński, is that comparison between distinct non-unique objects needs to be unspecified. This is so that you cannot ensure overlap. In other words:

```
constexpr char const* a = define_static_string("dedup");
constexpr char const* b = define_static_string("dup");

static_assert(b == b); // ok, #1
static_assert(b + 1 == b + 1); // ok, #2
static_assert(a != b); // ok, #3
static_assert(a + 2 != b); // error: unspecified
static_assert(string_view(a + 2) == string_view(b)); // ok, #4
```

Now, it had better be the case that `b == b` and `b + 1 == b + 1` are both valid checks. It would be fairly strange otherwise. Similarly, the goal is to not be able to observe whether `a + 2 == b`. It could be `true` at runtime. Or not. We have no idea at compile-time yet, so it'd be better to just not even allow an answer.

The interesting one is `a != b`. We could say that the comparison is unspecified (because they're pointers into distinct non-unique objects). But in this case, regardless of whether `a` and `b` overlap, `a != b` is *definitely* going to be `true` at runtime. So we should only make unspecified the case that we actually cannot specify. After all, it would be strange if `a == b` were unspecified but `a[0] == b[0]` was `false`.

Note that, regardless, the `string_view` comparison is valid, since that is comparing the contents.

This does present an interesting situation where `a == b` could be invalid but `is_same_v<C<a>, C<b>>` would be valid.

### § 3.1.1 Update from CWG

[\[CWG2765\]](#) now provides us a good solution for this problem. The wording (as of March 2025) introduces the idea that a pointer value pointing to a potentially non-unique object is associated with an evaluation — and that equality between pointers associated with different evaluations is non-constant.

We could simply add ourselves to the list of cases in that wording:

- ? A pointer value pointing to a potentially non-unique object  $O$  ([intro.object]) is *associated with* the evaluation of the *string-literal* ([lex.string]), or initializer list ([dcl.init.list]), or a call to either `std::define_static_string` or `std::define_static_array` ([??]) that resulted in the string literal object or backing array, respectively, that is  $O$  or of which  $O$  is a subobject. [ *Note 1*: A pointer value obtained by pointer arithmetic ([expr.add]) from a pointer value associated with an evaluation  $E$  is also associated with  $E$ . — *end note* ]

This would give us exactly the behavior stipulated above. `b == b` is associated with the same evaluation, so that's fine. But comparing `a + 2` to `b` would run afoul of the new rule that we're not allowed to evaluate

- (25.?) — an equality operator comparing pointers to potentially non-unique objects, if the pointer values of both operands are associated with different evaluations ([basic.compound]) and they can both point to the same offset within the same potentially non-unique object

Because `a + 2` *could be* `b`, it's non-constant. Problem solved.

## § 3.2 The Structural Question

For `define_static_string`, we have it easy because we know that `char` and `char8_t` are both structural types. But for `define_static_array`, we get an arbitrary `T`. How can we produce overlapping arrays in this case? If `T` is structural, we can easily ensure that equal invocations produce the *same* span result.

But if `T` is not structural, we have a problem, because `T*` is, regardless. So we have to answer the question of what to do with:

```
template <auto V> struct C { };

C<define_static_array(r).data()> c1;
C<define_static_array(r).data()> c2;
```

Either:

- this works, and it is unspecified whether `c1` and `c2` have the same type.
- the call to `define_static_array` works, but the resulting pointer is not usable as a non-type template argument (in the same way that string literals are not).
- the call to `define_static_array` mandates that the underlying type is structural.

None of these options is particularly appealing. The last prevents some very motivating use-cases since neither `span` nor `string_view` are structural types yet, which means you cannot reify a `vector<string>` into a `span<string_view>`, but hopefully that can be resolved soon ([P3380R1]). You can at least reify it into a `span<char const*>`?

For now, this paper proposes the last option, as it's the simplest (and the relative cost will hopefully decrease over time). Allowing the call but rejecting use as non-type template parameters is appealing though.

## § 3.3 Return Type and Layering

[ Editor's note: This section was migrated from [P3617R0] ]

define\_static\_array returning a `span<T const>` and `define_static_string` returning a `char const*` is useful, but sometimes it's insufficient. Matthias Wippich sent us some examples of such cases. Consider C++20 code such as:

```
template <size_t N>
struct FixedString {
    char data[N] = {};

    constexpr FixedString(char const(&str)[N]) {
        std::ranges::copy(str, str+N, data);
    }
};

template <FixedString S>
struct Test { };
```

This is a widely used pattern for being able to pass string literals as template arguments. However, there is no way to programmatically produce a string to pass in as an argument:

| Approach                                                                                        | Result                                                                                                                                     |
|-------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>using A = Test&lt;"foo"&gt;;</code>                                                       | ✓                                                                                                                                          |
| <code>using B = [: substitute(^^Test, {reflect_constant("foo"sv)}) :];</code>                   | ✗ Error: <code>std::string_view</code> isn't structural, but even if it was, this wouldn't work because you couldn't deduce the size of S. |
| <code>using C = Test&lt;define_static_string("foo")&gt;;</code>                                 | ✗ Error: cannot deduce the size of S                                                                                                       |
| <code>using D = [:substitute(^^Test, {reflect_constant(define_static_string("foo"))}):];</code> | ✗ Error: cannot deduce the size of S                                                                                                       |

The issue here is that `define_static_string` returns a `char const*`, which loses size information. If, instead, we had a lower layer function that returned a reflection of an array, we could easily use that:

| Approach                                                                         | Result |
|----------------------------------------------------------------------------------|--------|
| <code>using E = Test[:reflect_constant_string("foo"):];</code>                   | ✓      |
| <code>using F = [:substitute(^^Test, {reflect_constant_string("foo")}):];</code> | ✓      |

Another situation in which this comes up is in dealing with reflections of members. When you want to iterate over your members one-at-a-time, then `define_static_array` coupled with [\[P1306R4\]\\_\(Expansion\\_Statements\)](#) is perfectly sufficient:

```
template <class T>
auto f(T const& var) -> void {
    template for (constexpr auto M : define_static_array(nsdms(^^T))) {
        do_something_with(var.[:M:]);
    }
}
```

However, some situations require *all* the members at once. Such as in this [struct of arrays](#) implementation. `define_static_array` simply returns a `span`, but if we had a function that returned a reflection of an array, then combining [\[P1061R10\]\\_\(Structured Bindings can introduce a Pack\)](#) with [\[P2686R5\]\\_\(constexpr structured bindings and references to constexpr variables\)](#) lets us do this instead:

| Status Quo                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Proposed                                                                                                                                                                                                                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> template &lt;auto... V&gt; struct replicator_type { template&lt;typename F&gt;     constexpr auto operator&gt;&gt;(F body) const -&gt;         decltype(auto) {             return body.template operator()&lt;V...&gt;                 ();         } };  template &lt;auto... V&gt; replicator_type&lt;V...&gt; replicator = {};  constexpr auto     expand_all(std::span&lt;std::meta::info         const&gt; r)     -&gt; std::meta::info {     std::vector&lt;std::meta::info&gt; rv;     for (std::meta::info i : r) {         rv.push_back(reflect_value(i));     }     return substitute(^replicator, rv); }  template &lt;class T&gt; struct SoaVector {     // ...     // this is a span&lt;info const&gt;     static constexpr auto ptr_mems =         define_static_array(nsdms(^Pointers));     // ...      auto operator[](size_t idx) const -&gt; T {         return [: expand_all(ptr_mems) :] &gt;&gt;             [this, idx]&lt;auto... M&gt;{                 return T{pointers_.[:M:] [idx]...};             };     } }; </pre> | <pre> template &lt;class T&gt; struct SoaVector {     // ...     // reflection of an object of type info     // const[N]     static constexpr auto ptr_mems =         reflect_constant_array(nsdms(^Pointers));     // ...      auto operator[](size_t idx) const -&gt; T {         constexpr auto [...M] = [: ptr_mems :];         return T{pointers_.[:M:] [idx]...};     } }; </pre> |

With the reflection of an array object, we can directly splice it and use structured bindings to get the pack of members we need all in one go — without any layers of indirection. It’s a much more direct solution.

Having the higher level facilities is very useful, having this additional lower level facility would also be useful.

## § 3.4 Possible Implementation

`define_static_string` can be nearly implemented with the facilities in [\[P2996R7\]](#), we just need `is_string_literal` to handle the different signature proposed in this paper.

`define_static_array` is similar and simpler, so we’ll present that here:

```

namespace meta {
    template <typename T, T... Vs>
    inline constexpr T __fixed_array[sizeof...(Vs)]{Vs...};

    template <ranges::input_range R>
    constexpr auto reflect_constant_array(R&& r) -> info {

```

```

    auto args = vector<info>{^ranges::range_value_t<R>};
    for (auto&& elem : r) {
        args.push_back(reflect_constant(elem));
    }
    return substitute(^__fixed_array, args);
}

template <ranges::input_range R>
constexpr auto reflect_constant_string(R&& r) -> info {
    using T = ranges::range_value_t<R>;
    static_assert(std::same_as<T, char> or std::same_as<T, char8_t>);

    auto args = vector<info>{^ranges::range_value_t<R>};
    for (auto&& elem : r) {
        args.push_back(reflect_constant(elem));
    }

    // add a the null terminator, unless we're a string literal
    bool const add_null = [&]{
        if constexpr (requires { is_string_literal(r); }) {
            if (is_string_literal(r)) {
                return false;
            }
        }
        return true;
    }();

    if (add_null) {
        args.push_back(reflect_constant(T()));
    }
    return substitute(^__fixed_array, args);
}

}

template <ranges::input_range R>
constexpr auto define_static_string(R&& r)
    -> ranges::range_value_t<R> const*
{
    using T = ranges::range_value_t<R>;
    static_assert(std::same_as<T, char> or std::same_as<T, char8_t>);

    // produce the array
    auto array = meta::reflect_constant_string(r);

    // extract the pointer
    return extract<T const*>(array);
}

template <ranges::input_range R>
constexpr auto define_static_array(R&& r)
    -> span<ranges::range_value_t<R> const>
{
    using T = ranges::range_value_t<R>;

    // produce the array
    auto array = meta::reflect_constant_array(r);

    // turn the array into a span
    return span<T const>(extract<T const*>(array), extent(type_of(array)));
}

```



Note that this implementation gives the guarantee we talked about in the [previous section](#). Two invocations of `define_static_string` with the same contents will both end up returning a pointer into the same specialization of the (extern linkage) variable template `__array<V>`. We rely on the mangling of `V` (and `std::array` is a structural type if `T` is, which `char` and `char8_t` are) to ensure this for us. This won't ever produce overlapping arrays, would need implementation help for that, but it is a viable solution for all use-cases.

## § 3.5 Examples

### § 3.5.1 Use as non-type template parameter

```
template <const char *P> struct C { };

const char msg[] = "strongly in favor"; // just an idea..

C<msg> c1;                               // ok
C<"nope"> c2;                             // ill-formed
C<define_static_string("yay")> c3;       // ok
```

### § 3.5.2 Pretty-printing

In the absence of general support for non-transient `constexpr` allocation, such a facility is essential to building utilities like pretty printers.

An example of such an interface might be built as follow:

```
template <std::meta::info R> requires is_value(R)
constexpr auto render() -> std::string;

template <std::meta::info R> requires is_type(R)
constexpr auto render() -> std::string;

template <std::meta::info R> requires is_variable(R)
constexpr auto render() -> std::string;

// ...

template <std::meta::info R>
constexpr auto pretty_print() -> std::string_view {
    return define_static_string(render<R>());
}
```

This strategy [lies at the core](#) of how the Clang/P2996 fork builds its example implementation of the `display_string_of` metafunction.

### § 3.5.3 Promoting Containers

In the Jason Turner talk cited earlier, he demonstrates an [example](#) of taking a function that produces a `vector<string>` and promoting that into static storage, in a condensed way so that the function

```
constexpr std::vector<std::string> get_strings() {
    return {"Jason", "Was", "Here"};
}
```

Gets turned into an array of string views. We could do that fairly straightforwardly, without even needing to take the function `get_strings()` as a template parameter:

```
constexpr auto promote_strings(std::vector<std::string> vs)
    -> std::span<std::string_view const>
{
    // promote the concatenated strings to static storage
    std::string_view promoted = std::define_static_string(
        std::ranges::fold_left(vs, std::string(), std::plus()));

    // now build up all our string views into promoted
    std::vector<std::string_view> views;
    for (size_t offset = 0; std::string const& s : vs) {
        views.push_back(promoted.substr(offset, s.size()));
        offset += s.size();
    }

    // promote our array of string_views
    return std::define_static_array(views);
}

constexpr auto views = promote_strings(get_strings());
```

Or at least, this will work once `string_view` becomes structural. Until then, this can be worked around with a `structural_string_view` type that just has public members for the data and length with an implicit conversion to `string_view`.

### § 3.5.4 With Expansion Statements

Something like this — [\[P1306R4\]\(Expansion Statements\)](#) — is not doable without non-transient `constexpr` allocation :

```
constexpr auto f() -> std::vector<int> { return {1, 2, 3}; }

constexpr void g() {
    template for (constexpr int I : f()) {
        // doesn't work
    }
}
```

But if we promote the contents of `f()` first, then this would work fine:

```
constexpr void g() {
    template for (constexpr int I : define_static_array(f())) {
        // ok!
    }
}
```

And the lower layer would let us [expand the whole pack](#) in one go:

```
constexpr void g() {
    constexpr auto [...m] = [: reflect_constant_array(f()) :];
    static_assert((... + m) == 6);
}
```

### § 3.5.5 Implementing `source_location`

One interesting use of a specific `define_static_object` (for the single object case), courtesy of Richard Smith, is to implement the single-pointer optimization for `std::source_location` without compiler support:

```
class source_location {
    struct impl {
        char const* filename;
        int line;
    };
    impl const* p_;

public:
    static constexpr auto current(char const* file = __builtin_FILE(),
                                  int line = __builtin_LINE()) noexcept
        -> source_location
    {
        // first, we canonicalize the file
        impl data = {.filename = define_static_string(file), .line = line};

        // then we canonicalize the data
        impl const* p = define_static_object(data);

        // and now we have an external linkage object mangled with this location
        return source_location{p};
    }
};
```

## § 3.6 Related Papers in the Space

A number of other papers have been brought up as being related to this problem, so let's just enumerate them.

- [\[P3094R5\]](#) ([std::basic\\_fixed\\_string](#)) proposed `std::basic_fixed_string<char, N>`. It exists to solve the problem that `C<"hello">` needs support right now. Nothing in this paper would make `C<"hello">` work, although it might affect the way that you would implement the type that makes it work.
- [\[P3380R1\]](#) ([Extending support for class types as non-type template parameters](#)) proposes to extend non-type template parameter support, which could eventually make `std::string` usable as a non-type template parameter. But without non-transient `constexpr` allocation, this doesn't obviate the need for this paper. Note that that paper even depends on this paper for how to normalize string literals, making string literals usable as non-type template arguments.
- [\[P1974R0\]](#) and [\[P2670R1\]](#) propose approaches to tackle the non-transient allocation problem.

Given non-transient allocation *and* a `std::string` and `std::vector` that are usable as non-type template parameters, this paper likely becomes unnecessary. Or at least, fairly trivial:

```
template <auto V>
inline constexpr auto __S = V.c_str();

template <ranges::input_range R>
constexpr auto define_static_string(R&& r) -> ranges::range_value_t<R> const* {
    using T = ranges::range_value_t<R>;
    static_assert(std::same_as<T, char> or std::same_as<T, char8_t>);

    auto S = ranges::to<basic_string<T>>(r);
    return extract<T const*>(substitute(^^__S, {meta::reflect_value(S)}));
}
```

## § 4 Wording

Change 6.7.2 [\[intro.object\]](#) to add the results of `reflect_constant_array` and `reflect_constant_string` as being potentially non-unique:

- 9 An object is a *potentially non-unique object* if it is
- (9.1) — a string literal object ([lex.string]),
  - (9.2) — the backing array of an initializer list ([dcl.init.ref]),
  - (9.3) — the object introduced by a call to `std::meta::reflect_constant_array` or `std::meta::reflect_constant_string` ([meta.reflection.array]), or
  - (9.4) — a subobject thereof.

Update the wording in [\[CWG2765\]](#) to account for these as well, in 6.8.4 [\[basic.compound\]/x](#):

- ? A pointer value pointing to a potentially non-unique object *O* ([intro.object]) is *associated with* the evaluation of the *string-literal* ([lex.string]), ~~or~~ initializer list ([dcl.init.list]), or a call to either `std::meta::reflect_constant_string` or `std::meta::reflect_constant_array` ([meta.reflection.array]) that resulted in the string literal object or backing array, respectively, that is *O* or of which *O* is a subobject. [ *Note 1*: A pointer value obtained by pointer arithmetic ([expr.add]) from a pointer value associated with an evaluation *E* is also associated with *E*. — end note ]

No change to 7.6.10 [\[expr.eq\]](#) is necessary, [\[CWG2765\]](#) takes care of it.

Add to [\[meta.reflection.synop\]](#):

```
#include <initializer_list>

- namespace std::meta {
+ namespace std {
+   namespace std {
+     // [meta.string.literal], checking string literals
+     constexpr bool is_string_literal(const char* p);
+     constexpr bool is_string_literal(const wchar_t* p);
+     constexpr bool is_string_literal(const char8_t* p);
+     constexpr bool is_string_literal(const char16_t* p);
+     constexpr bool is_string_literal(const char32_t* p);

+     // [meta.define.static], promoting to static storage
+     template <ranges::input_range R>
+       constexpr const ranges::range_value_t<R>* define_static_string(R&& r);
+
+     template <ranges::input_range R>
+       constexpr span<const ranges::range_value_t<R>> define_static_array(R&& r);
+
+     template <class T>
+       constexpr const remove_cvref_t<T>* define_static_object(T&& r);
+
+ namespace meta {
+   using info = decltype(^::);
+   // ...

+   // [meta.reflection.result], expression result reflection
+   template<class T>
+     constexpr info reflect_value(const T& value);
+   template<class T>
+     constexpr info reflect_object(T& object);
+   template<class T>
```

```

    consteval info reflect_function(T& fn);

+   // [meta.reflection.array], promoting to static storage
+   template <ranges::input_range R>
+   consteval info reflect_constant_string(R&& r);
+
+   template <ranges::input_range R>
+   consteval info reflect_constant_array(R&& r);

    // ...
+ }
}

```

Add the new subclause [meta.reflection.array] “Promoting to Static Storage”:

- 1 The functions in this subclause promote compile-time storage into static storage.
 

```

        template <ranges::input_range R>
        consteval info reflect_constant_string(R&& r);
      
```
- 2 Let *CharT* be `ranges::range_value_t<R>`.
- 3 *Mandates:* *CharT* is one of `char`, `wchar_t`, `char8_t`, `char16_t`, or `char32_t`.
- 4 Let *V* be the pack of values of type *CharT* whose elements are the corresponding elements of *r*, except that if *r* refers to a string literal object, then *V* does not include the trailing null terminator of *r*.
- 5 Let *P* be the template parameter object ([temp.param]) of type `const CharT[sizeof...(V)+1]` initialized with `{V..., CharT() }`.
- 6 *Returns:*  $\wedge\wedge P$ .
- 7 [ *Note 2:* *P* is a potentially non-unique object ([intro.object]). — *end note* ]
 

```

        template <ranges::input_range R>
        consteval info reflect_constant_array(R&& r);
      
```
- 8 Let *T* be `ranges::range_value_t<R>`.
- 9 *Mandates:* *T* is a structural type ([temp.param]), `is_constructible_v<T, ranges::range_reference_t<R>>` is `true`, and `is_copy_constructible_v<T>` is `true`.
- 10 *Constant When:* `reflect_constant(e)` is a constant subexpression for every element *e* of *r*.
- 11 Let *V* be the pack of values of type `info` of the same size as *r*, where the *i*<sup>th</sup> element is `reflect_constant(ei)`, where *e<sub>i</sub>* is the *i*<sup>th</sup> element of *r*.
- 12 Let *P* be
  - (12.1) — If `sizeof...(V) > 0`, then the template parameter object ([temp.param]) of type `const T[sizeof...(V)]` initialized with `{[:V:],...}`.
  - (12.2) — Otherwise, the template parameter object of type `array<T, 0>` initialized with `{}`.
- 13 *Returns:*  $\wedge\wedge P$ .
- 14 [ *Note 3:* *P* is a potentially non-unique object ([intro.object]). — *end note* ]

Add to the new subclause [meta.string.literal]:

```

constexpr bool is_string_literal(const char* p);
constexpr bool is_string_literal(const wchar_t* p);
constexpr bool is_string_literal(const char8_t* p);
constexpr bool is_string_literal(const char16_t* p);
constexpr bool is_string_literal(const char32_t* p);

```

1 *Returns:*

- (1.1) — If *p* points to an unspecified object ([*expr.const*]), *false*.
- (1.2) — Otherwise, if *p* points to a subobject of a string literal object ([*lex.string*]), *true*.
- (1.3) — Otherwise, *false*.

Add to the new subclause [meta.define.static], “Promoting to Static Storage”

1 The functions in this clause promote compile-time storage into static storage.

```

template <ranges::input_range R>
constexpr const ranges::range_value_t<R>* define_static_string(R&& r);

```

2 *Effects:* Equivalent to:

```

return extract<const ranges::range_value_t<R>*>(meta::reflect_constant_string(r));

template <ranges::input_range R>
constexpr span<const ranges::range_value_t<R>> define_static_array(R&& r);

```

3 *Effects:* Equivalent to:

```

using T = ranges::range_value_t<R>;
meta::info array = meta::reflect_constant_array(r);
if (is_array_type(type_of(array))) {
    return span<const T>(extract<const T*>(array), extent(type_of(array)));
} else {
    return span<const T>();
}

```

```

template <class T>
constexpr const remove_cvref_t<T>* define_static_object(T&& t);

```

4 *Effects:* Equivalent to:

```

using U = remove_cvref_t<T>;
if constexpr (is_class_type(^U)) {
    return addressof(extract<const U&>(meta::reflect_constant(std::forward<T>(t))));
} else {
    return define_static_array(span(addressof(t), 1)).data();
}

```

5 [ *Note 4:* For class types, *define\_static\_object* provides the address of the template parameter object ([*temp.param*]) that is template-argument-equivalent to *t*. — *end note* ]

## § 4.1 Feature-Test Macro

Add to 17.3.2 [version.syn]:

```

#define __cpp_lib_define_static 2025XX // freestanding, also in <meta>

```

## § 5 Acknowledgements

---

Thanks to Matthias Wippich for the insightful observation that led to the additions of `meta::reflect_constant_array` and `meta::reflect_constant_string`.

## § 6 References

---

[CWG2765] CWG. 2023-07-14. Address comparisons between potentially non-unique objects during constant evaluation.

<https://cplusplus.github.io/CWG/issues/2765.html>

[P1061R10] Barry Revzin, Jonathan Wakely. 2024-11-24. Structured Bindings can introduce a Pack.

<https://wg21.link/p1061r10>

[P1306R4] Dan Katz, Andrew Sutton, Sam Goodrick, Daveed Vandevoorde, Barry Revzin. 2025-04-24. Expansion Statements.

<https://wg21.link/p1306r4>

[P1974R0] Jeff Snyder, Louis Dionne, Daveed Vandevoorde. 2020-05-15. Non-transient constexpr allocation using `prop-const`.

<https://wg21.link/p1974r0>

[P2484R0] Richard Smith. 2021-11-17. Extending class types as non-type template parameters.

<https://wg21.link/p2484r0>

[P2670R1] Barry Revzin. 2023-02-03. Non-transient constexpr allocation.

<https://wg21.link/p2670r1>

[P2686R5] Corentin Jabot, Brian Bi. 2024-11-12. constexpr structured bindings and references to constexpr variables.

<https://wg21.link/p2686r5>

[P2996R7] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, Dan Katz. 2024-10-13. Reflection for C++26.

<https://wg21.link/p2996r7>

[P3094R5] Mateusz Pusz. 2024-10-15. `std::basic_fixed_string`.

<https://wg21.link/p3094r5>

[P3380R1] Barry Revzin. 2024-12-04. Extending support for class types as non-type template parameters.

<https://wg21.link/p3380r1>

[P3491R0] Barry Revzin, Peter Dimov, Daveed Vandevoorde, Dan Katz. 2024-12-15. `define_static_{string,object,array}`.

<https://wg21.link/p3491r0>

[P3491R1] Barry Revzin, Wyatt Childers, Peter Dimov, Daveed Vandevoorde. 2025-01-13. `define_static_{string,object,array}`.

<https://wg21.link/p3491r1>

[P3491R2] Barry Revzin, Wyatt Childers, Peter Dimov, Daveed Vandevoorde. 2025-03-14. `define_static_{string,object,array}`.

<https://wg21.link/p3491r2>

[P3617R0] Barry Revzin. 2025-05-16. `std::meta::reflect_constant_{array,string}`.

<https://wg21.link/p3617r0>